

Prompt Caching 命中攻略

OpenRouter + Anthropic 模型適用

緩存讀取 0.1x 成本 2026-07 更新

讓每次請求的開頭盡量跟上一次一模一樣，就能從緩存裡讀，省 90% 輸入成本。

Anthropic 的緩存是從你請求的第一個 token 開始，一個一個往後比。跟上次一樣的部分，直接從緩存讀，只收十分之一的錢。一旦遇到不一樣的地方——斷了，後面全部原價。

所以你要做的就是一件事：**把不會變的東西堆在前面，會變的東西全部塞到最後面。**

具體來說就是三層。第一層是 system message，放人設、指令、工具定義，打上緩存標記，永遠不動，每輪都命中。第二層是對話歷史，越聊越長，但舊的部分不會變，在最後一條用戶消息的前一條掛一個斷點，告訴 Anthropic「到這裡為止都跟上次一樣」。每輪這個斷點往前滾一格。第三層是最新的用戶消息，時間、記憶、狀態這些每輪都變的東西全部注入在這裡。這條不緩存——但它是整個請求的最後一條，所以它變了不影響前面兩層。

再加一個 session_id，確保 OpenRouter 每次把請求送到同一個節點。因為緩存是節點級別的，換節點就沒了。

就這樣。下面是技術細節。

成本一覽

正常輸入

1.0x

無緩存

首次寫入

1.25x

建立緩存

緩存命中

0.1x

TTL 內讀取

第一次請求多付 25% 寫入費，之後每次命中省 90%。TTL 預設 5 分鐘，可延長至 1 小時。對話場景下第二輪就能回本。

💡 核心原理：前綴匹配

緩存是從頭比對的

Anthropic 的緩存機制是**前綴匹配**——從請求的第一個 token 開始往後比對，一旦遇到不一樣的內容，後面全部失效。開頭不能動，變化的東西往後放。

靜態 / 動態分離

把 prompt 拆兩塊：**靜態部分**（人設、指令、工具定義）放 system message，打上 cache_control；**動態部分**（時間、記憶、狀態）注入最新一條 user message。這樣 system 永遠不變，每輪都命中。

動態內容絕對不進 system

如果你把時間戳、記憶摘要、任何每輪會變的東西塞進 system message，system 的 hash 每輪都變，前綴匹配直接斷掉——**不只 system 的緩存失效，連帶後面所有對話歷史的緩存也全部失效**。動態的東西永遠注入到最新的 user message 裡。

📐 消息佈局

system	靜態 prompt（人設 / 指令 / 工具定義）	✓
user[0]	第一條用戶消息	✓
asst[0]	第一條回覆	✓
...	對話歷史（越聊越長，但舊的不變）	✓
asst[n-1]	上一條回覆 ←rolling breakpoint	✓
user[n]	[動態注入]+ 最新用戶輸入	✗

最新的 user message 包含動態內容，每輪都變，不緩存。但它在最後面，所以它變了不影響前面所有已緩存的前綴。

實作三步

① system message 加 cache_control ▲

把 system 的 content 從字串改成 blocks 陣列，加上 cache_control：

```
{
  "role": "system",
  "content": [{
    "type": "text",
    "text": "<你的靜態 prompt>",
    "cache_control": {
      "type": "ephemeral",
      "ttl": "1h"
    }
  }]
}
```

ttl 設 "1h" 把緩存有效期從預設 5 分鐘延長到 1 小時。對聊天場景很重要——用戶可能隔幾分鐘才回消息。

② rolling breakpoint 緩存對話歷史 ▲

在最新 user message 的**前一條**消息上掛 cache_control。每輪對話這個斷點往前滾一格：

```
// 找到倒數第二條消息（最新 user 的前一條）
// 把它的 content 轉成 blocks 格式
// 在最後一個 block 上加：
lastBlock["cache_control"] = {
  "type": "ephemeral",
  "ttl": "1h"
}
```

為什麼不掛在最新 user message 上？因為它包含動態注入的內容，每輪都變，掛了也白掛。

③ session_id 黏性路由 ▲

OpenRouter 會把請求分散到多個後端節點。緩存只在同一個節點上生效，所以要用 session_id 把同一段

對話釘在同一個節點：

```
{
  "model": "anthropic/claude-sonnet-4",
  "messages": [...],
  "session_id": "conv_abc123"
}
```

一個對話一個 `session_id`，不是一個 app 一個。不同對話的歷史不同，緩存前綴本來就不會匹配。

! 五個常見踩坑

× 動態內容塞進 system message

時間戳、記憶摘要、狀態放進 system → system 每輪都變 → 前綴匹配直接斷掉 → system 緩存失效，連帶歷史對話緩存也全部失效。動態的東西永遠注入到最新 user message。

× 用 provider.order 鎖路由

有人用 provider.order 指定後端節點來代替黏性路由——這會破壞 OpenRouter 的路由機制。用 session_id 就好。

× 用頂層 automatic cache_control

OpenRouter 支援頂層 "cache_control": "automatic"，但它會啟發式地放斷點，經常放到最新 user message 上。下一輪這條消息進入歷史，內容變了（動態注入的部分不見了），hash 永遠不匹配，緩存永久失效。手動放 block-level 斷點才可控。

× 沒有開關控制

寫入成本 1.25x。如果 system prompt 很短或對話歷史很少，寫入開銷可能大於節省。加一個用戶端開關，關掉時發純字串格式，不帶 cache_control 和 session_id。

× blocks 格式的類型問題

掛 cache_control（一個 Map/Object）到 content block 上時，如果你的語言用了嚴格類型（比如 Dart 的 Map<String, String>），cache_control 的值是 Map 不是 String，會 runtime 報錯。用 Map<String, dynamic> 或等價的寬泛類型。

驗證緩存命中

檢查 response 裡的 usage 欄位：

prompt_tokens	總輸入 token (含緩存的)
cache_creation_input_tokens	本次寫入緩存的 token (1.25x)
cache_read_input_tokens	本次從緩存讀取的 token (0.1x)

健康模式：第一條消息 `cache_creation > 0` (寫入)，第二條開始 `cache_read > 0` 且 `cache_creation = 0` 或很小 (rolling breakpoint 移動)。如果 `cache_read` 一直是 0，回去看踩坑清單。



Streaming 注意

streaming 模式下，緩存統計在最後一個 chunk 的 usage 裡。加上這個確保拿到：

```
"stream_options": { "include_usage": true }
```



上線清單

- ✓ system prompt 拆成靜態 (cacheable) 和動態 (per-turn) 兩部分
- ✓ 靜態部分用 content blocks 格式 + cache_control，ttl 設 "1h"
- ✓ 動態部分注入最新 user message，不進 system
- ✓ rolling breakpoint 掛在最新 user message 的前一條
- ✓ 每個對話用獨立的 session_id 做黏性路由
- ✓ 提供用戶端開關，可關閉緩存
- ✓ streaming 加 stream_options.include_usage
- ✓ 上線後看 cache_read_input_tokens 確認命中

